Course: Advanced Databases

Assignment: Phase 1 - Data Model

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

1. Application Domain

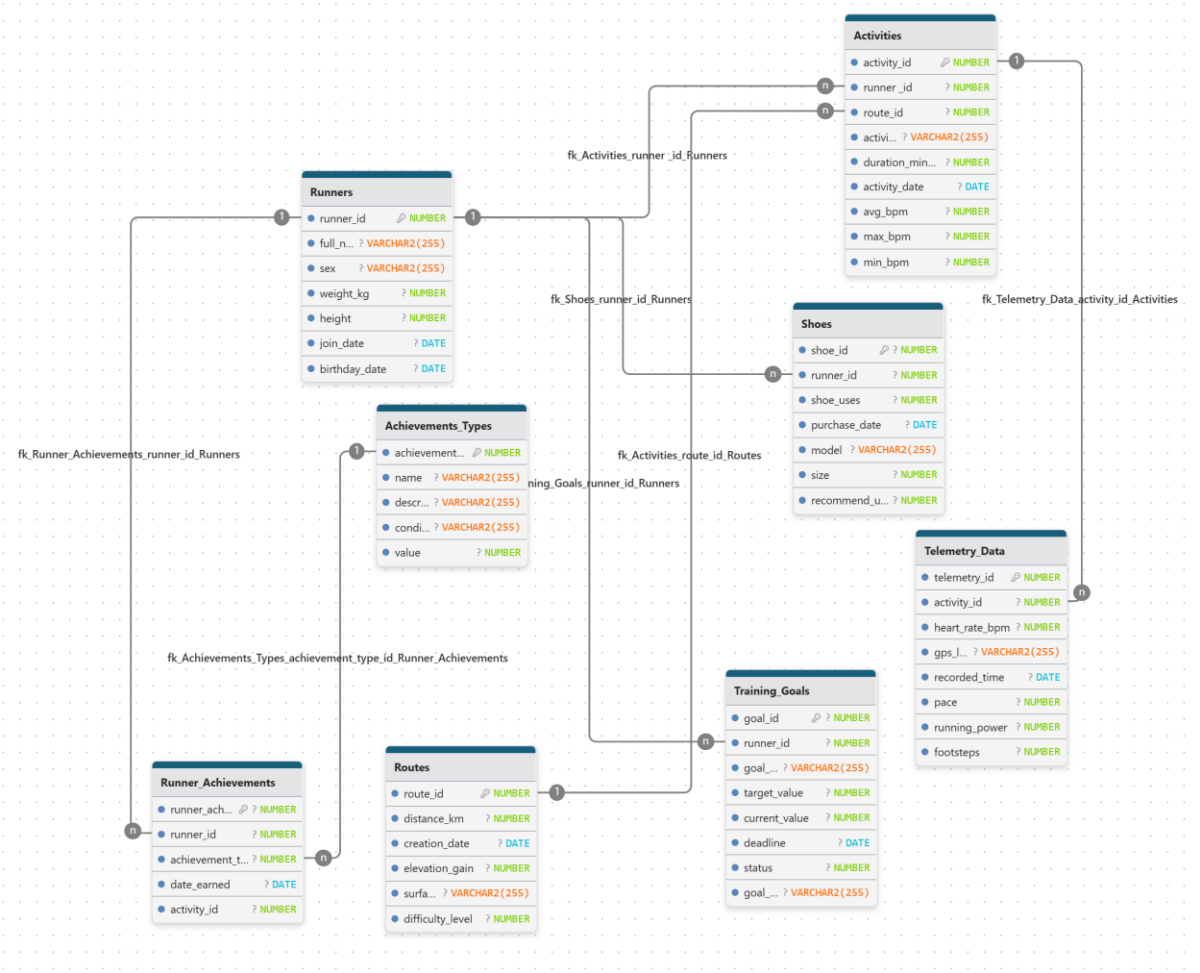
Topic: Running Tracking System

Description: A database system for a running application that records users' profiles, their overall workout sessions, and telemetry data such as heart rate. It also tracks the lifecycle of the runners' shoes, routes, and earned achievements.

2. Data Model Diagram & Specification

Entity-Relationship Overview (Associations):

- One **Runner** can have many **Shoes**, **Activities**, **Training_Goals**, and **Runner_Achievements** (1-to-Many).
- One **Route** can be used in many **Activities** (1-to-Many).
- One **Activity** contains many points of **Telemetry_Data** (1-to-Many).
- One **Activity** can result in many **Runner_Achievements** (1-to-Many).
- One **Achievement_Type** can be awarded as many **Runner_Achievements** (1-to-Many).



Assignment: Phase 2 - Workload

1. Workload Description

This workload simulates the real-world operational demands of the Running Tracking System. It consists of complex transactions designed to guarantee observable execution times for performance tuning. The workload is divided into two categories: complex querying operations that simulate analytical reporting across multiple joined relations, and data-changing operations that handle heavy telemetry data usage and background data recalculations.

2. Query operations.

- **First query operation.**

  - **Description:** This query shows runners together with their total distance covered in a given year (2026). It returns only those runners whose total distance is greater than the average total distance covered by all runners in that year.
  - **Database action:** The query displays each runner's ID and full name along with a calculated column representing the total distance in kilometers. The data is obtained by joining three tables: Runners, Activities, and Routes. Only activities from the year 2026 are considered using a date filter. The results are grouped by runner ID and full name to calculate the total distance per runner. A subquery is used to compute the average total distance across all runners, and only those runners whose total distance exceeds this average are included. Finally, the results are sorted from highest to lowest total distance.

```
SELECT
   r.runner_id, r.full_name,
   SUM(ro.distance_km) AS total_distance_km
FROM
Runners r JOIN Activities a
   ON r.runner_id = a.runner_id
JOIN Routes ro
   ON a.route_id = ro.route_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
GROUP BY
   r.runner_id,
```

```
  r.full_name
HAVING SUM(ro.distance_km) > (
  SELECT AVG(total_distance)
  FROM (
    SELECT
      SUM(ro2.distance_km) AS total_distance
    FROM Activities a2
    JOIN Routes ro2
      ON a2.route_id = ro2.route_id
    WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
    GROUP BY a2.runner_id
  )
)
ORDER BY total_distance_km DESC;
```

- **Second query operation.**

  - **Description:** This query displays the runner ID, full name, total distance covered, and number of achievements for the top 10 runners whose total distance in 2026 is greater than the average distance covered by all runners in that year.
  - **Database action:** The database joins the Runners, Activities, and Routes tables to calculate the total distance covered by each runner based on the routes assigned to their activities. It also performs a left join with the Runner_Achievements table to count the number of achievements for each runner. Only activities from the year 2026 are considered using a date filter. The data is then grouped by runner ID and full name to compute aggregated values such as total distance and number of achievements. A subquery is used to calculate the average total distance covered by all runners in 2026, and only those runners whose total distance exceeds this average are included in the result. Finally, the results are sorted in descending order of total distance, and only the top 10 runners are returned.

```
SELECT
  r.runner_id, r.full_name,
  SUM(ro.distance_km) AS total_distance_km,
  COUNT(DISTINCT ra.runner_achievement_id) AS total_achievements
FROM Runners r
JOIN Activities a
```

```
   ON r.runner_id = a.runner_id
JOIN Routes ro
   ON a.route_id = ro.route_id
LEFT JOIN Runner_Achievements ra
   ON r.runner_id = ra.runner_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
GROUP BY
   r.runner_id,
   r.full_name
HAVING SUM(ro.distance_km) > (
   SELECT AVG(total_distance)
   FROM (
      SELECT
         SUM(ro2.distance_km) AS total_distance
      FROM Activities a2
      JOIN Routes ro2
         ON a2.route_id = ro2.route_id
      WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
      GROUP BY a2.runner_id
   )
)
ORDER BY total_distance_km DESC
FETCH FIRST 10 ROWS ONLY;
```

- **Third query operation**

  - **Description:** This query displays runner IDs, full names, and the number of activities for runners whose average heart rate in 2026 is lower than the average heart rate calculated across all runners. It includes only runners who have completed at least 5 activities.
  - **Database action:** The database joins the Runners, Activities, and Telemetry_Data tables to connect runners with their activity records and heart rate measurements. It filters data for the year 2026 and groups results by runner to calculate the average heart rate and number of activities. A subquery computes the overall average heart rate across all runners. The final result includes only those runners whose average heart rate is below this value and who have completed at least 5 activities.

```
SELECT
  r.runner_id, r.full_name,
  AVG(t.heart_rate_bpm) AS avg_heart_rate,
  COUNT(DISTINCT a.activity_id) AS total_activities
FROM Runners r
JOIN Activities a
  ON r.runner_id = a.runner_id
JOIN Telemetry_Data t
  ON a.activity_id = t.activity_id
WHERE EXTRACT(YEAR FROM a.activity_date) = 2026
GROUP BY
  r.runner_id,
  r.full_name
HAVING COUNT(DISTINCT a.activity_id) >= 5
  AND AVG(t.heart_rate_bpm) < (
    SELECT AVG(runner_avg_hr)
    FROM (
      SELECT
        AVG(t2.heart_rate_bpm) AS runner_avg_hr
      FROM Activities a2
      JOIN Telemetry_Data t2
        ON a2.activity_id = t2.activity_id
      WHERE EXTRACT(YEAR FROM a2.activity_date) = 2026
      GROUP BY a2.runner_id
      HAVING COUNT(DISTINCT a2.activity_id) >= 5
    )
  )
ORDER BY avg_heart_rate ASC;
```

3. Changing operations

- The **INSERT** operation: "Syncing a Smartwatch Workout"

    - **Description:** When a user finishes a run, the app saves the workout.
    - **Database Action:** This transaction inserts a single summary row into the **Activities** table. Immediately after, in the same transaction, it inserts hundreds of individual rows into the **Telemetry_Data** table to record the runner's **gps_location** (like "Park Szczytnicki in Wrocław"), **heart_rate_bpm**, **pace**, and **running_power** at various **recorded_time** intervals throughout that single activity.

- The **UPDATE** operation: "Nightly Training Goal Recalculation"

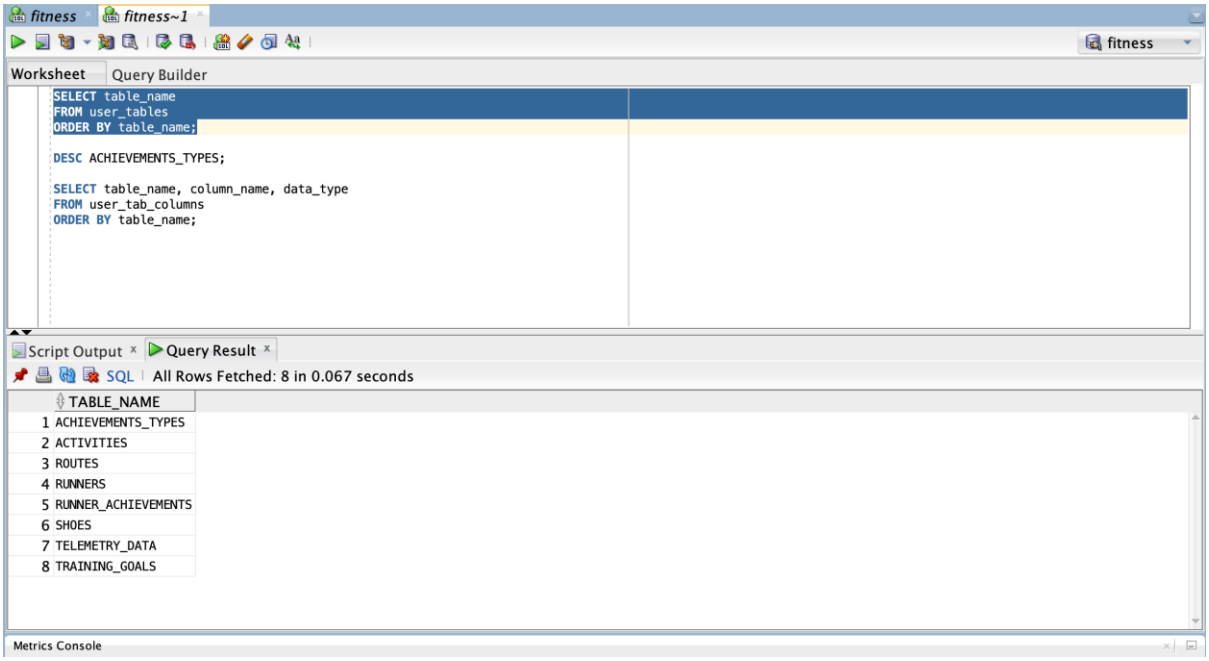
    - **Description:** Every night, the application runs a background job to check the progress of every runner's active training goals (e.g., "Run 100km this month").
    - **Database Action:** This transaction uses an **UPDATE** statement on the **Training_Goals** table. It recalculates the **current_value** for users by looking at their recent **Activities** and updates the **status** column (e.g., changing it from '0' for In Progress to '1' for Completed) if they hit their **target_value**.

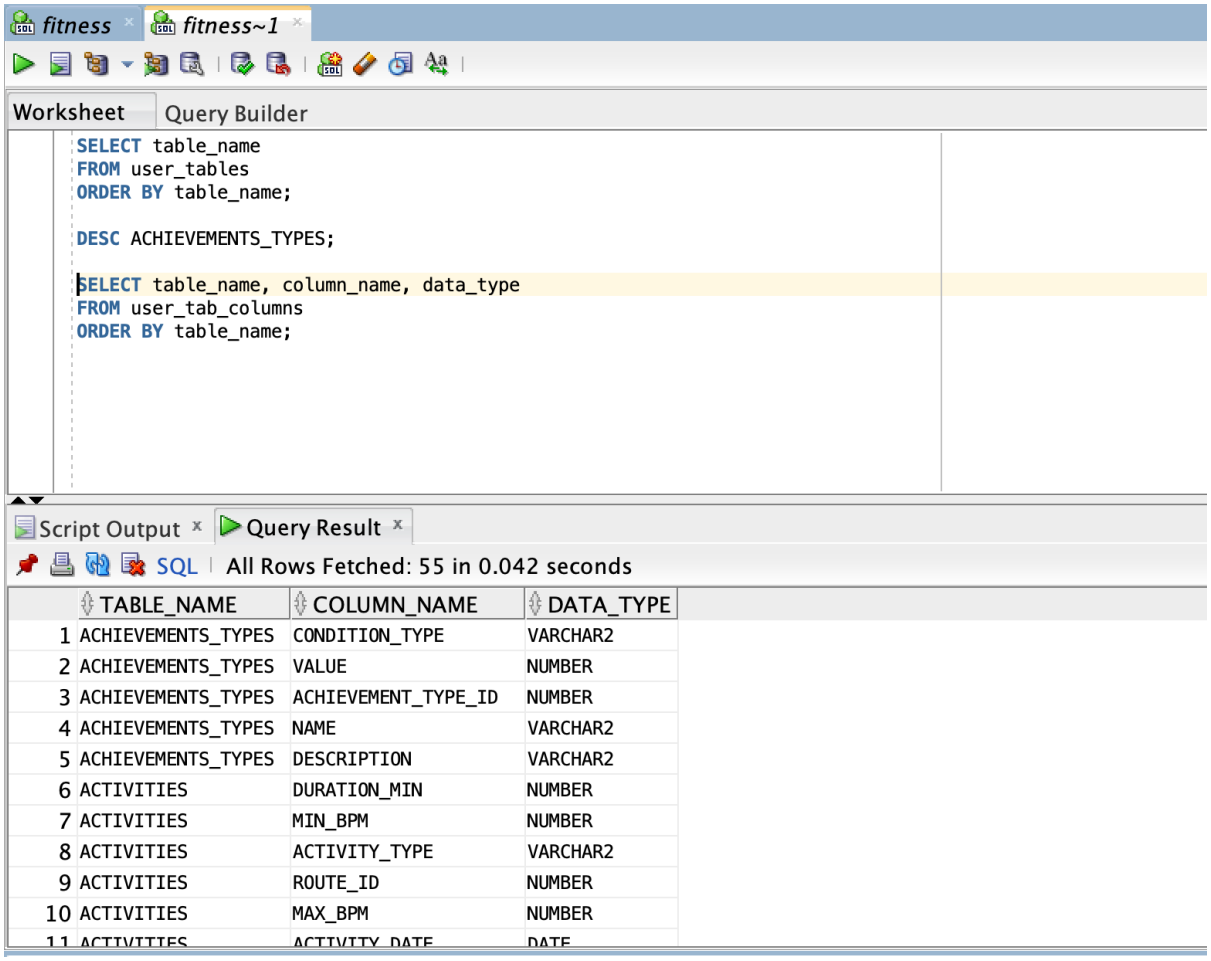
- The **DELETE** Transaction: "Account Deletion"

    - **Description:** A runner decides to permanently delete their account and all associated fitness data from the app.
    - **Database Action:** This transaction is a very big, multi-step **DELETE** operation. Because of the **Referential Integrity,** the database cannot just delete the user from the **Runners** table first! The transaction must delete data in a specific order: first wiping out thousands of rows of **Telemetry_Data** and then their **Runner_Achievements**, then their **Activities**, **Shoes**, and **Training_Goals**, before finally using a **DELETE** statement on the Runners table itself.

Assignment: Phase 3 – DBMS

The database was created using Oracle Database 21c Express Edition, following the professor's recommendation on eportal. A dedicated user schema was created, and the database structure was implemented using SQL DDL statements.




fitness
fitness~1
Worksheet
Query Builder
SELECT table_name
FROM user_tables
ORDER BY table_name;
DESC ACHIEVEMENTS_TYPES;
SELECT table_name, column_name, data_type
FROM user_tab_columns
ORDER BY table_name;
Script Output
Query Result
SQL
All Rows Fetched: 55 in 0.042 seconds
TABLE_NAME
COLUMN_NAME
DATA_TYPE
1 ACHIEVEMENTS_TYPES CONDITION_TYPE VARCHAR2
2 ACHIEVEMENTS_TYPES VALUE NUMBER
3 ACHIEVEMENTS_TYPES ACHIEVEMENT_TYPE_ID NUMBER
4 ACHIEVEMENTS_TYPES NAME VARCHAR2
5 ACHIEVEMENTS_TYPES DESCRIPTION VARCHAR2
6 ACTIVITIES DURATION_MIN NUMBER
7 ACTIVITIES MIN_BPM NUMBER
8 ACTIVITIES ACTIVITY_TYPE VARCHAR2
9 ACTIVITIES ROUTE_ID NUMBER
10 ACTIVITIES MAX_BPM NUMBER